

-
- # Employee
- ## *Management System*

A modular C programming project with file persistence and activity logging

`main.c`

`employee_operations.
c`

`file_handler.c`

`utils.c`

`employee.h`

Presented by Thupten Dolkar

WHAT IT DOES



A console-based system to manage employee records

Add Employee

Store ID, name, age, salary, position

Display All

View every saved employee record

Search

Find by Employee ID or Name

Modify

Update any field for existing employee

Delete

Remove a record from the system

Export

Write all records to employees.txt

Auto-Save

Every change saved to employees.dat

Activity Log

Every action logged to activity_log.txt

MODULAR FILE STRUCTURE



employee.h

Shared Header

- Struct definition
- All #define constants
- All function prototypes
- Included by every .c file



main.c

Entry Point

- Global array + counter
- loadFromFile() on start
- Menu loop (do-while)
- Calls other functions



employee_operations.c

CRUD Logic

- addEmployee()
- displayEmployees()
- searchEmployee()
- modifyEmployee()
- deleteEmployee()



file_handler.c

File I/O — • saveToFile() binary · •
loadFromFile() binary · •
exportEmployees() txt · • logAction() with
time



utils.c

Helpers — • clearBuffer() · •
printEmployee() · • printSeparator()

FILE HANDLING & LOGGING

Three files are automatically created and managed by the program

BINARY FILE

employees.dat

Stores all employee records in binary format. Written by fwrite(), read back by fread() on every startup.

🕒 Auto-saved after every Add, Modify, Delete

TEXT EXPORT

employees.txt

Human-readable copy of all records. Created when user selects option 5 from the menu.

🕒 Created on demand — Menu option 5

ACTIVITY LOG

activity_log.txt

Timestamped record of every user action. Uses time.h to stamp each line automatically.

🕒 Appended after every single action

Sample activity_log.txt output:

```
[2025-04-24 14:30:01] STARTUP      | Data loaded from employees.dat
[2025-04-24 14:31:05] ADD         | Thupten Dolkar
[2025-04-24 14:32:18] SEARCH      | Found by ID
[2025-04-24 14:33:44] DELETE      | Ram Bahadur
```

HOW FILE SAVING WORKS

fwrite saves data — fread brings it back — always in the same order



Program Starts

main() calls
loadFromFile()



File Opens

fopen("employees.dat",
"rb") → read binary



fread()

1. Read count
2. Read all structs



In Memory (RAM)

employees[] array
numEmployees = N



SAVE (fwrite)

```
fwrite(&numEmployees,  
      sizeof(int), 1, file);  
fwrite(employees,  
      sizeof(struct Employee),  
      numEmployees, file);
```



LOAD (fread)

```
fread(&numEmployees,  
      sizeof(int), 1, file);  
fread(employees,  
      sizeof(struct Employee),  
      numEmployees, file);
```

CHALLENGES & SOLUTIONS

01

scanf + fgets Bug

⚠ Leftover `\n` in stdin caused fgets to return empty string instantly.

✓ `clearBuffer()` in `utils.c` flushes stdin after every scanf call.

02

Binary File I/O

⚠ Never used fread/fwrite before. Didn't know how to save a struct array.

✓ Write count first, then array. fread reverses it exactly.

03

Duplicate IDs

⚠ Two employees with same ID made search and delete unreliable.

✓ Loop checks array before adding — rejects duplicate ID immediately.

04

Splitting into Files

⚠ Moving functions to separate .c files caused 'undefined reference' errors.

✓ `employee.h` declares all prototypes — every file includes this header.

05

Adding Timestamps to Log

⚠ Didn't know how to get the current date and time in C.

✓ Used `time.h`: `time()`, `localtime()`, `strftime()` to format timestamps.

06

Compiling Multiple Files

⚠ Typing gcc with all 4 file names was long and easy to get wrong.

✓ Makefile handles it — just type: `make`

WHAT I LEARNED

Key concepts taken from building this project



Struct + Header Files

The struct lives in `employee.h` — included everywhere. Change it once, all files update.



Modular Design

One job per file. `employee_operations.c` never touches files — that is `file_handler.c`'s job.



Timestamps with `time.h`

`time()` → `localtime()` → `strftime()` gives formatted date-time for the activity log.



Binary File I/O

`fread/fwrite` save structs exactly as-is in memory. Always write count first.



The `scanf` Newline Trap

`scanf` leaves `\n` in `stdin`. Without `clearBuffer()`, `fgets` reads empty. Always flush.



Makefile

`gcc` with 4 files is one command in the Makefile. Make only recompiles changed files.

HOW TO COMPILE & RUN

Using the Makefile — one command does everything

1

Open Terminal in VS Code

↳ *Terminal → New Terminal*

```
$ Ctrl + ` (backtick key)
```

2

Go to your project folder

↳ *Or drag folder into terminal*

```
$ cd FILES-2
```

3

Compile all files

↳ *Builds: main.c + all .c files*

```
$ make
```

4

Run the program

↳ *The menu will appear*

```
$ ./employment
```

Bonus: `make clean` → removes all .o files for a fresh build

Project Summary

- 5 separate files — each with one clear responsibility
- 3 output files: `employees.dat`, `employees.txt`, `activity_log.txt`
- `fread` / `fwrite` for binary persistence — data survives restarts
- Timestamped activity log written automatically on every action
- Makefile compiles all 4 source files with a single: `make`
- Full CRUD: Add, Display, Search, Modify, Delete, Export

THANK YOU